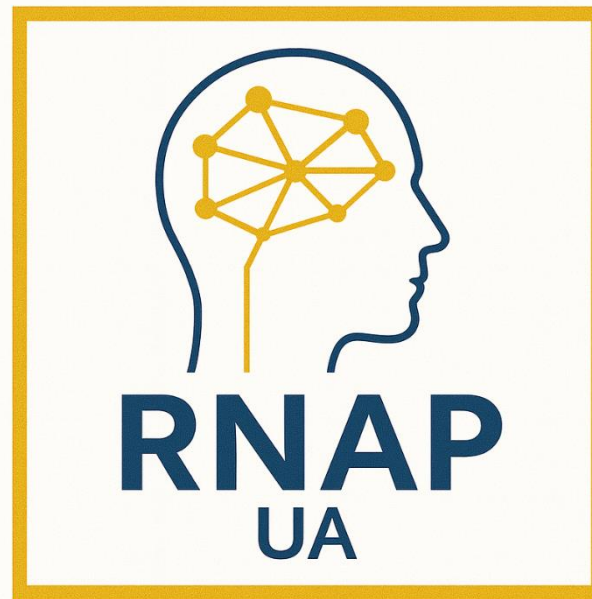
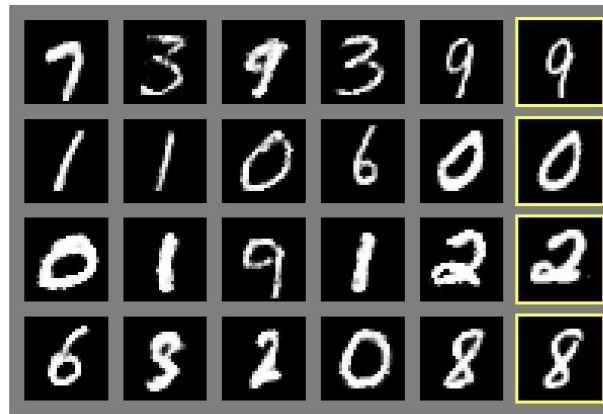


Redes Neuronales y Aprendizaje Profundo



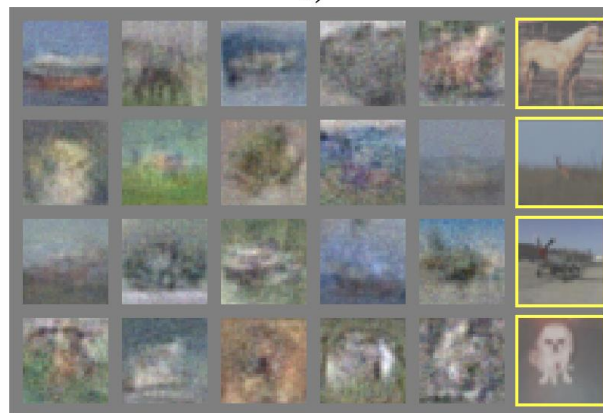
Bloque5: Generative Adversarial Networks



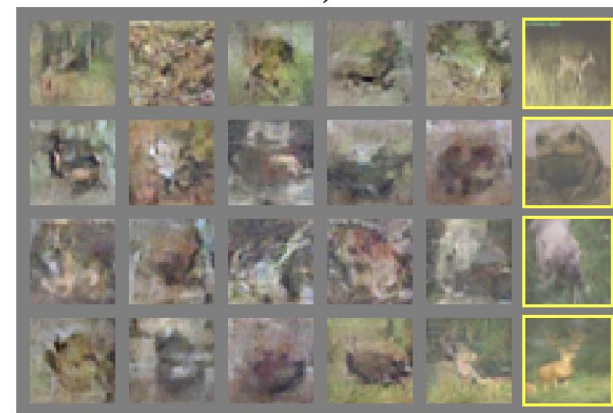
a)



b)



c)



d)

- **Objectives**

- Understand **what a generative model is** and why GANs can compete with other alternatives such as Diffusion Models and VAEs.
- Describe the **GAN architecture**: the role of the **Generator (G)** and **Discriminator (D)**, their inputs and outputs.
- Interpret the minimax loss function and explain the **Nash equilibrium** the training process converges toward.

- **Objectives**

- Distinguish the main GAN variants — **DCGAN**, **cGAN**, and **CycleGAN** — and know when to apply each.
- Identify the main **training failure modes** (mode collapse, instability) and the most widely used solutions (WGAN, spectral normalization).
- Evaluate the quality of a generative model using metrics such as **FID** and **Inception Score**.

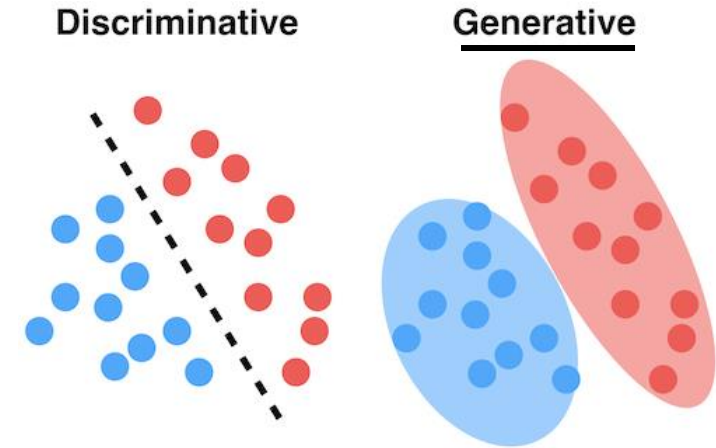


Contents

- Motivation: generative models (general advantages and limitations).
- The GAN framework: Generator, Discriminator, and the adversarial game.
- The minimax objective function and its theoretical foundations.
- The alternating training loop: step-by-step optimization of D and G.
- DCGAN: convolutional architecture, batch normalization, and activation functions.
- Conditional GAN (cGAN): conditioning on labels or images, Pix2Pix.
- CycleGAN: unpaired image-to-image translation and cycle-consistency loss
- Common training failures: mode collapse, vanishing gradients, and instability.
- Solutions and stabilization techniques: WGAN, spectral normalization, minibatch discrimination.
- Evaluation metrics: FID, Inception Score, Precision and Recall.

Motivation and Context

Generative models learn the distribution of data $p(x)$ to generate new realistic samples. The main generative paradigms include Variational Autoencoders (VAEs), Diffusion Models and Generative Adversarial Networks (GANs).



- **VAE** — Encodes data into a structured latent space and reconstructs it; stable to train but tends to produce blurry outputs.
- **GAN** — Pits a generator against a discriminator to produce sharp, realistic samples; high visual quality but training is unstable and prone to mode collapse.
- **Diffusion** — Learns to reverse a gradual noising process achieving state-of-the-art quality and diversity; but inference is slow due to the many denoising steps required.

Motivation and Context

GAN - Goodfellow et al. (2014)

- Original Mini max Framework - FC layers - MNIST

cGAN - Mirza et al. (2014)

- Conditioned generation

Pix2Pix - Isola et al. (2016)

- Paired image translation

CycleGAN - Zhu et al. (2017)

- Unpaired Image Translation

ProGAN - Karra et al. (2017)

- Progressive growing (1024 x 1024)

StyleGAN - Karra et al. (2018)

- Style-based generator

StyleGAN2 - Karra et al. (2019)

GigaGAN - Kang et al. (2023)

DCGAN - Radford et al. (2014)

- Convolution layers

InfoGAN - Chen et al. (2016)

- Disentangled representations

WGAN - Arjovsky et al. (2017)

- Solves Vanishing Gradients

WGAN-GP - Gulrajani et al. (2017)

- Improved WGAN

BigGAN - Brock et al. (2018)

- Large-scale conditioned

FID and Precision/Recall

StyleGAN3 - Karra et al. (2021)

Motivation and Context



Motivation and Context

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Examples of discriminative metrics.

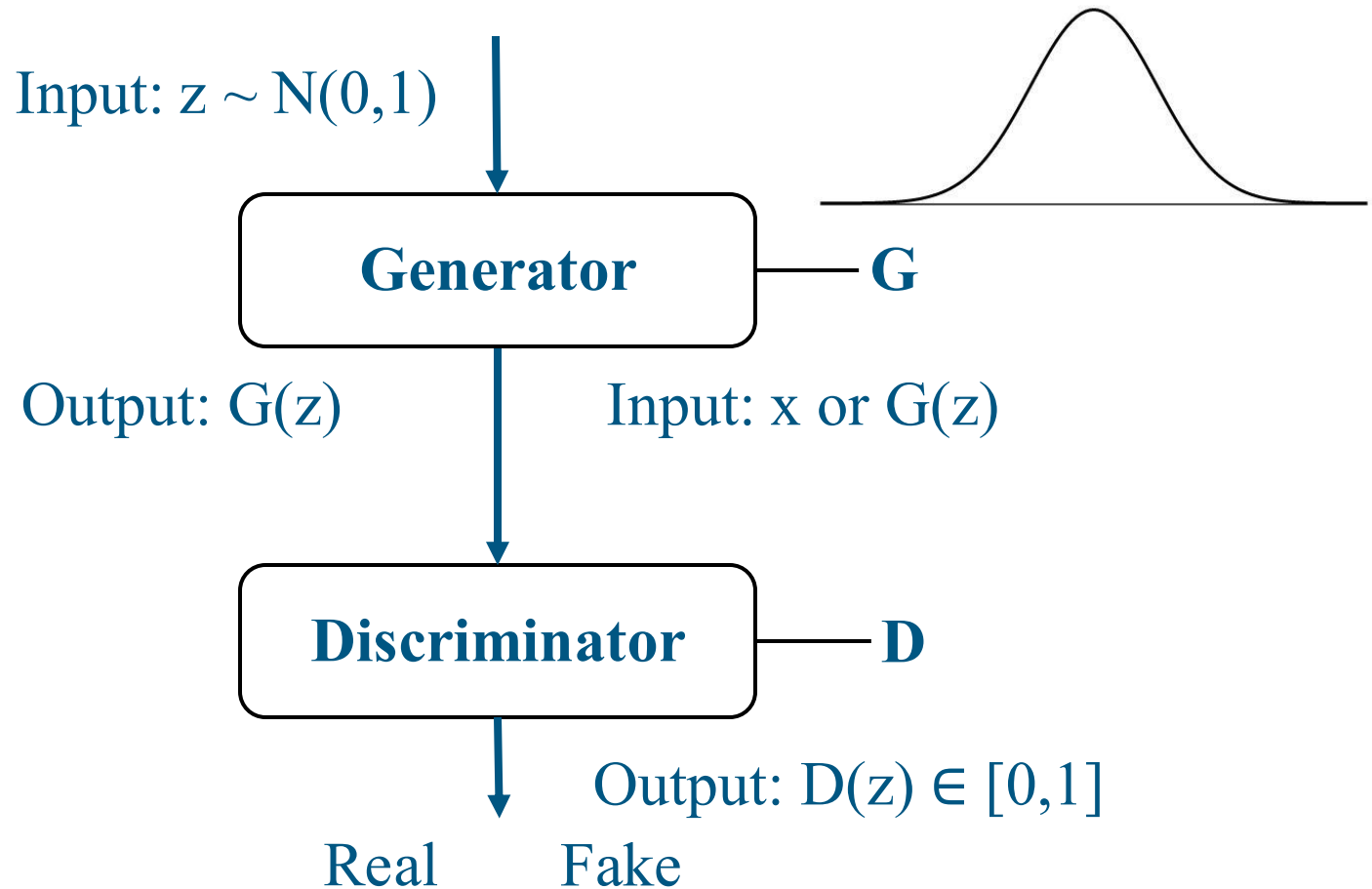
The objective is to classify a sample or predict a number.

Examples of generative metrics. The objective is to generate quality samples derived from the dataset distribution.

Human evaluation

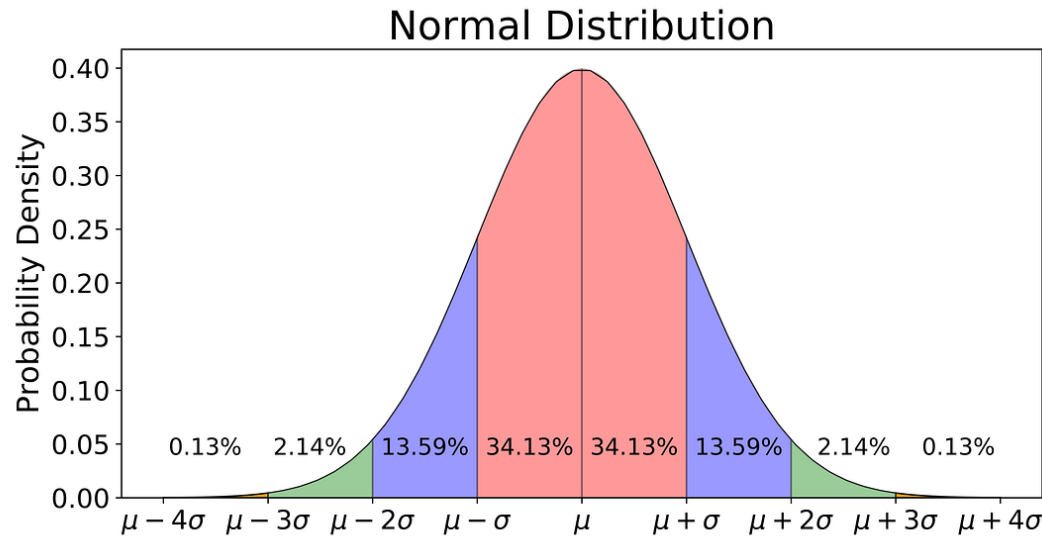
$$FID = \|\mu - \mu_w\|^2 + \text{tr} \left(\Sigma + \Sigma_w - 2(\Sigma \Sigma_w)^{\frac{1}{2}} \right)$$

The GAN Framework



"It is an adversarial game where the generator tries to fool the discriminator, and the discriminator tries to classify the input between a real sample and a generated one."

Normal Distribution



- The normal distribution **covers all of $\mathbb{R}^n$ with no gaps**, so the generator has no "dead zones" in the latent space.
- **Any distribution can be obtained** by transforming a normal through a sufficiently expressive function, and a deep network qualifies.
- $N(0,1)$ is isotropic: **all latent directions are treated equally**, which is what makes **interpolation and vector arithmetic work**.
- In practice, **other distributions (uniform, spherical) also work**, the choice barely matters as long as the network is expressive enough.

Minimax Objective

Minimax Equation

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

"D tries to maximize the value; G tries to minimize it"

$$E_{x \sim p_{data}(x)} [\log D(x)]$$

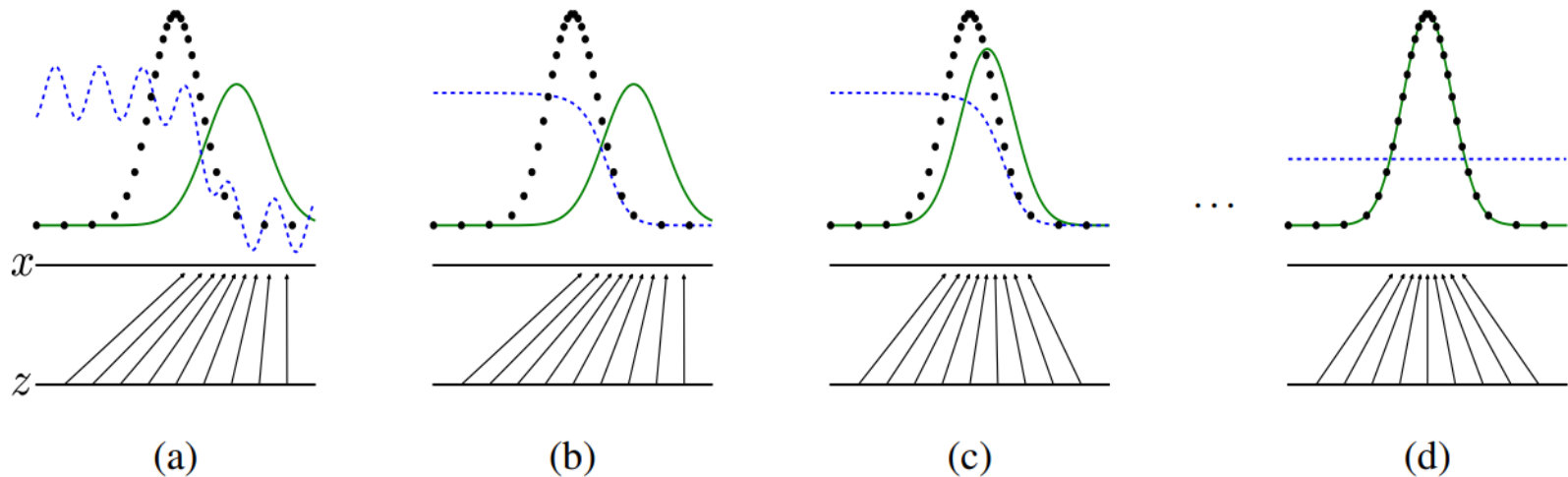
D receives a real sample x and outputs $D(x)$, its estimated probability that x is real. D wants this to be close to 1, so $\log D(x) \rightarrow 0$ (maximum). If D is fooled and outputs 0.5, $\log(0.5) = -0.69$, which is worse for D.

$$E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

G generates a fake sample $G(z)$ from random noise z . D receives it and outputs $D(G(z))$. D wants this close to 0 (fake), so $1 - D(G(z)) \rightarrow 1$ and $\log(1 - D(G(z))) \rightarrow 0$ (maximum). G wants the opposite: it wants $D(G(z)) \rightarrow 1$, making this term very negative (minimum).

Nash Equilibrium

The game reaches equilibrium when neither player can improve by changing their strategy alone. This happens when $D(G(z)) = 0.5$ for every generated sample where the discriminator is essentially flipping a coin because the fake samples are indistinguishable from real ones. At that point, G has learned the true data distribution and there is nothing left to exploit.



Discriminative distribution - Data distribution - Generative distribution

Training Loop (Algorithm)

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log \left(1 - D(G(z^{(i)})) \right) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(z^{(i)})) \right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Training Loop (Visualization)

Step 1.

We define:

- Real_label as 1
- Fake_label as 0

1

Original

0

Generated

Training Loop (Visualization)

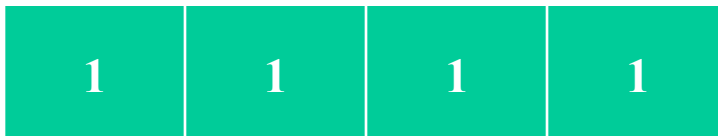
Step 2.

We get:

- The `batch_size` defined in our trainloader

We create:

- One vector of `real_labels` (ones) with the same size of `batch_size`
- One vector of `fake_labels` (zeros) with the same size of `batch_size`



Step 3.

We fed:

- The discriminator

With what?

- The real images.
- And we compute the Loss (Binary) and the gradients.

BinaryCrossEntropy?

- Yes

1 0

Training Loop (Visualization)

Step 4.

We generate:

- Noise from batch_size and noise_dim (a vector of vectors)

Then, we generate:

- Fake (or generated images) from the previous noise
- And we compute the Loss (Binary) and the gradients.

Then:

- Fed the samples to the discriminator and get the output labels.
- We compute the loss (yes, again) and the gradients.
- We average this gradients and the previous ones.
- Optimizer step for the discriminator.

We finished the discriminator training (this epoch)!

Training Loop (Visualization)

Step 5.

We fed:

- The discriminator with fake (generated) images and get the output labels.

Then:

- We compute the generator los between the ouput and real labels (ones).
- We calculate gradients and optimizar step for the generator.

We finished the generator training (this epoch)!

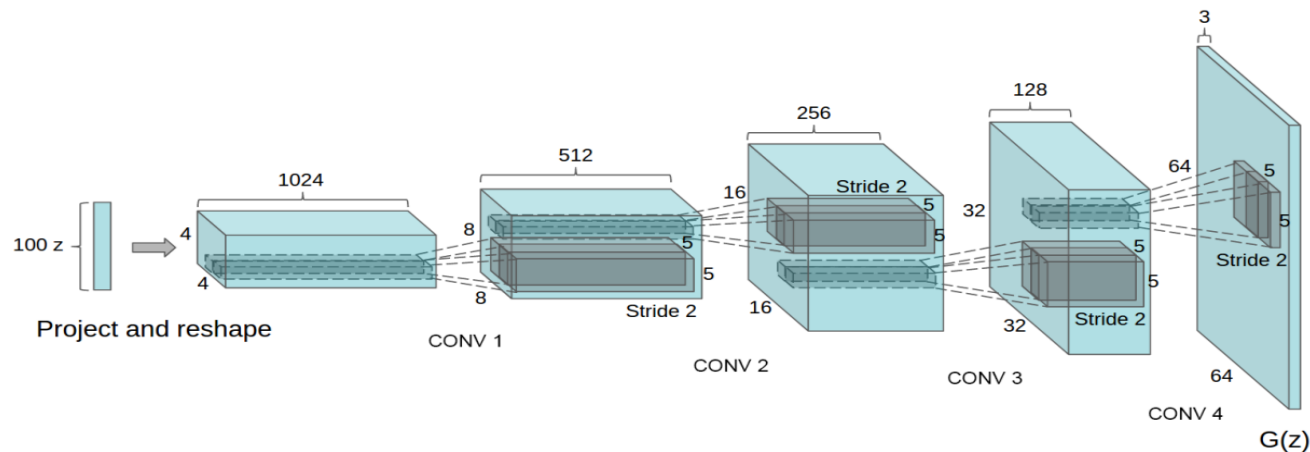
Just repeat steps 1-5 (n epochs)



Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks

Architecture guidelines for stable Deep Convolutional GANs

- Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Use batchnorm in both generator and the discriminator.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in generator for all layers except for the output, which uses Tanh.
- Use LeakyReLU activation in the discriminator for all layers.



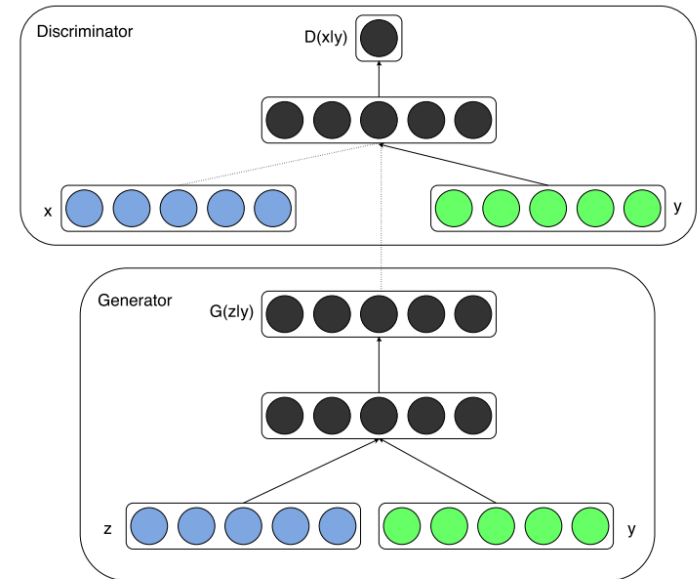
Conditional Generative Adversarial Nets

"Generative adversarial nets can be extended to a conditional model if both the generator and discriminator are conditioned on some extra information y ".

This extra information can be represented as:

- Labels (encoded as one-hot or embedding).
- Embeddings from other modalities.

This extra information can be combined in joint hidden representations or spatial concatenation.

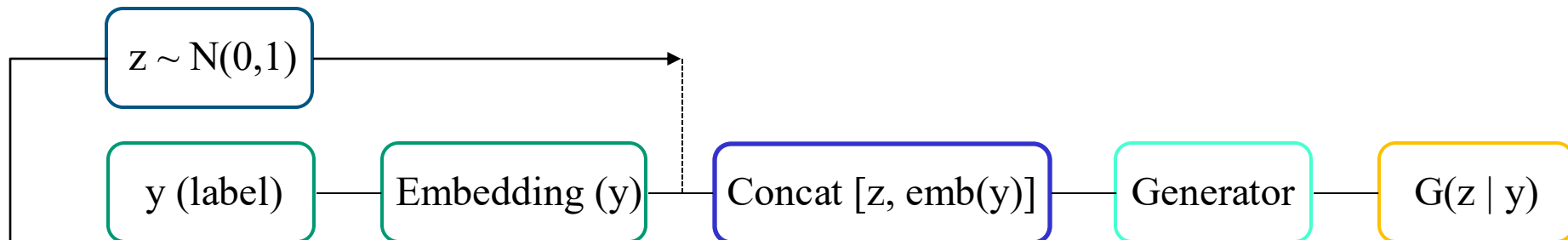


Minmax Equation

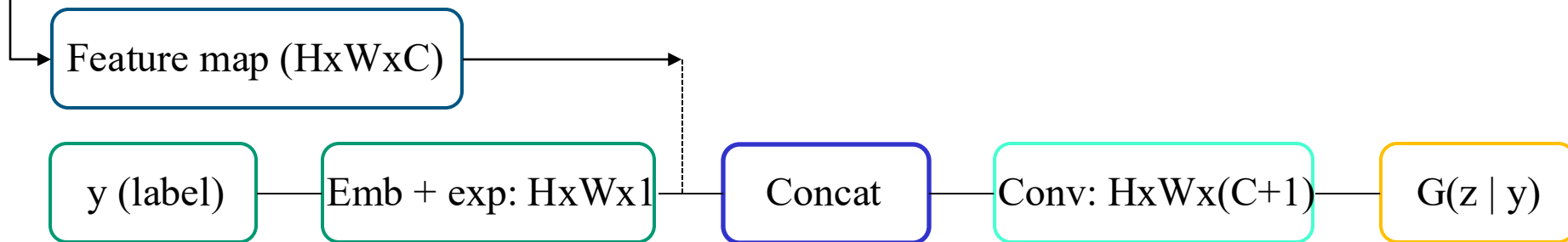
$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x | y)] + E_{z \sim p_z(z)} \left[\log (1 - D(G(z | y))) \right]$$

cGAN (Embedding and Spatial concatenation)

Embedding-level concatenation (fast, lower cost, less spatial control)



Spatial-level concatenation (fine per-pixel control, more parameters)



cGAN (Embedding and Spatial concatenation)

Spatial-level concatenation (fine per-pixel control, more parameters)

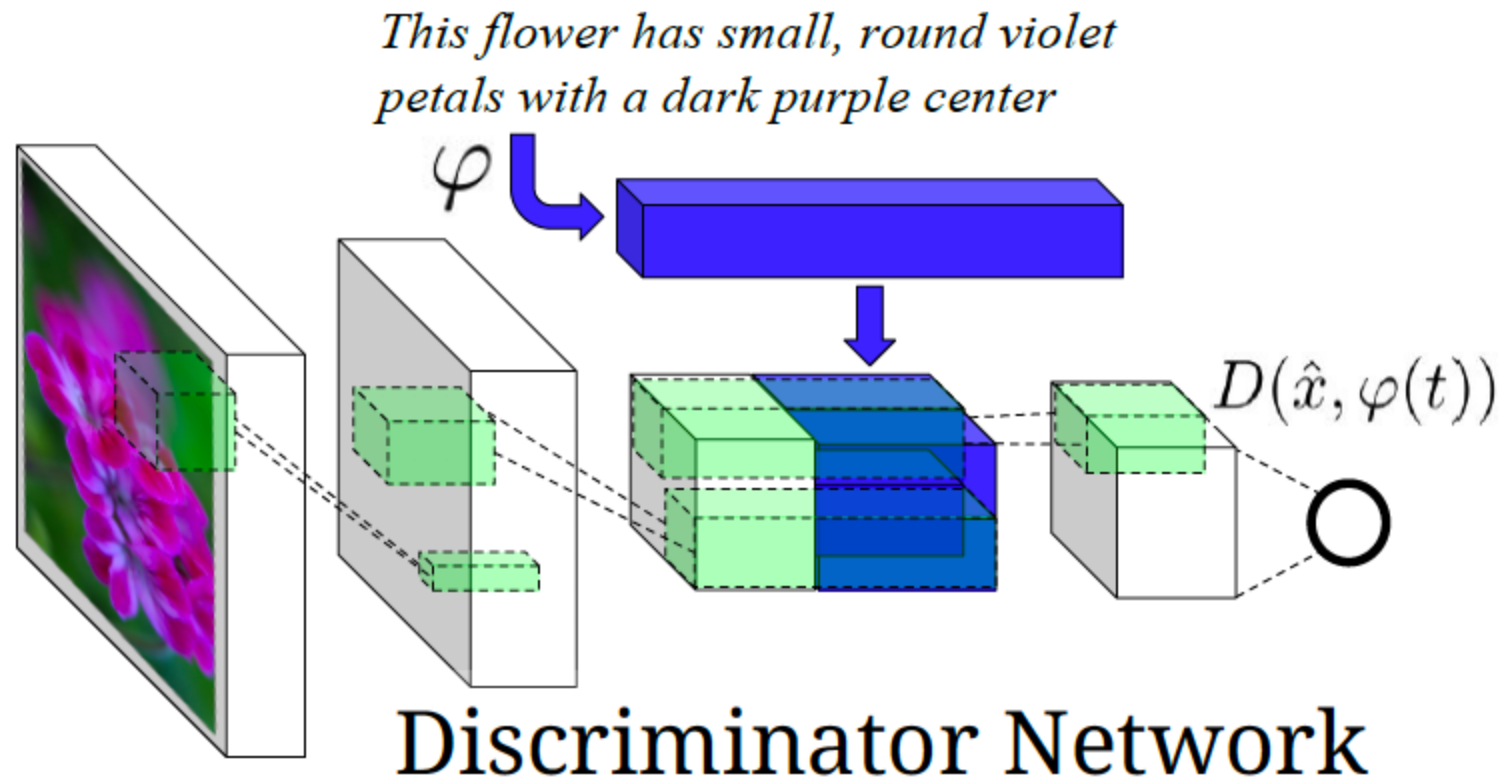
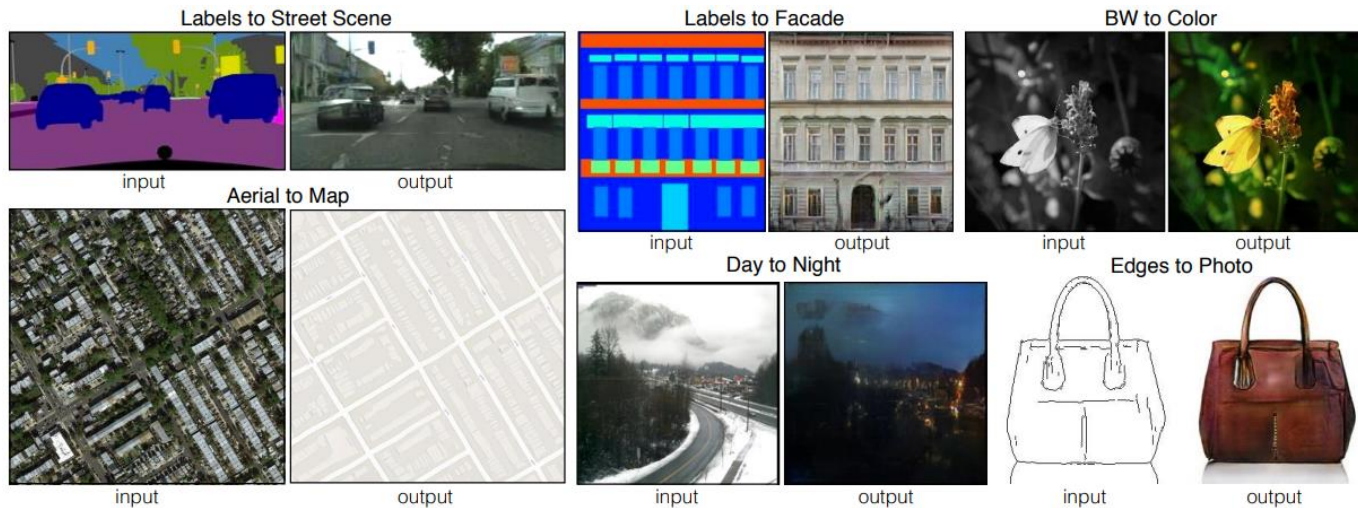
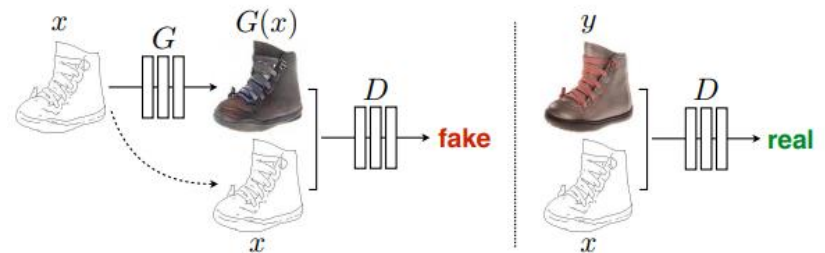


Figure from: <https://towardsdatascience.com/overview-of-gans-conditioning-methods-e4eac018a7f3/>

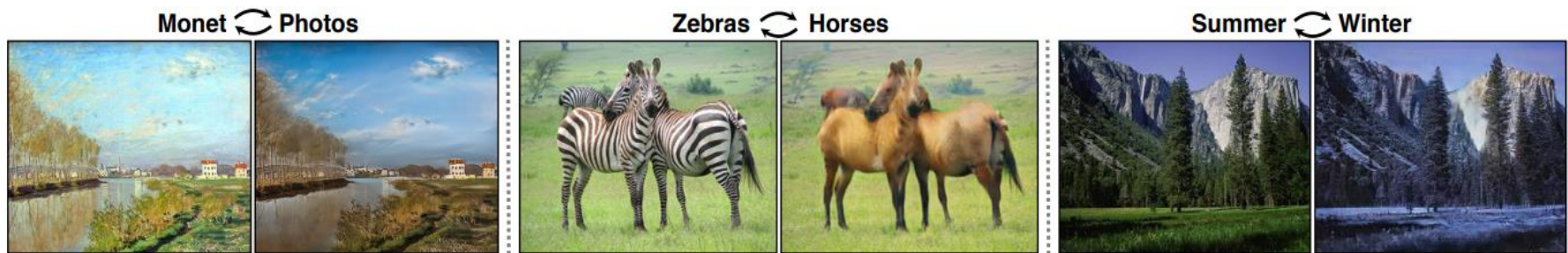
Image-to-Image Translation with Conditional Adversarial Networks



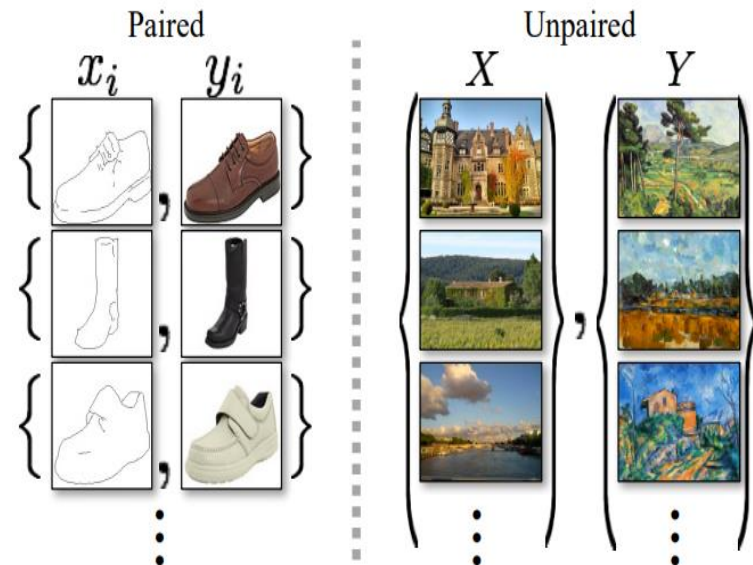
The **Generator** takes the **Edge image** as the input. To be able to apply this methodology, they use a **U-Net based neural network**. They also use an L2 los for G.



Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks



CycleGAN learns to **translate images between two unpaired domains X and Y** — for example horses and zebras — without needing matched image pairs for training. **It does this by training two generators simultaneously: G maps $X \rightarrow Y$ and F maps $Y \rightarrow X$,** each paired with its own discriminator that tries to distinguish real images from translated ones. The key insight is the cycle-consistency loss: if you translate a horse into a zebra and then translate that zebra back, you should recover the original horse — enforcing $F(G(x)) \approx x$ and $G(F(y)) \approx y$. This constraint forces the generators to preserve the content and structure of the input rather than just producing any plausible-looking image in the target domain, making meaningful translation possible without supervision.



Point Cloud GAN

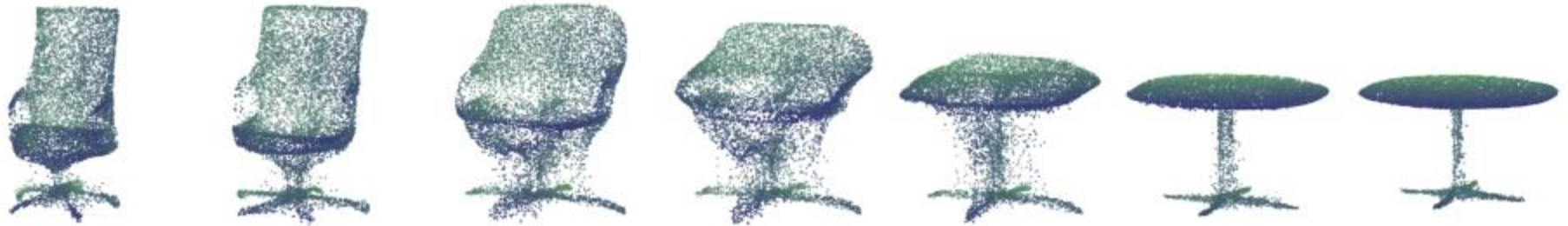


Figure 9: Interpolating between a table and a chair point clouds, using our latent space representation.

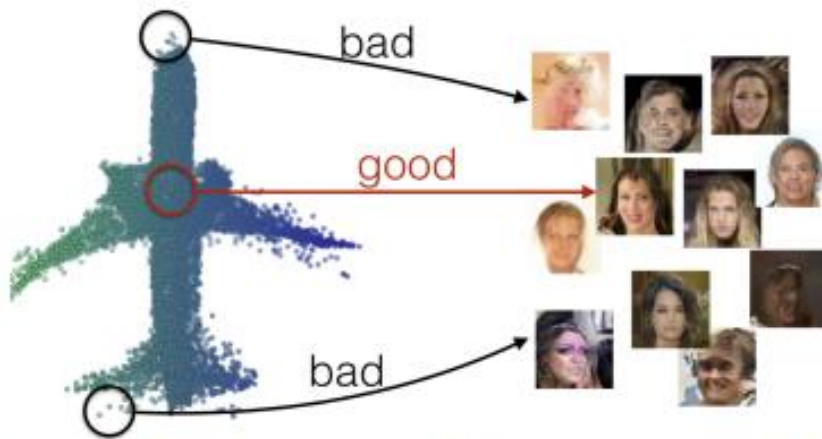


Figure 4: Connection between good/bad points and faces generated from a GAN.

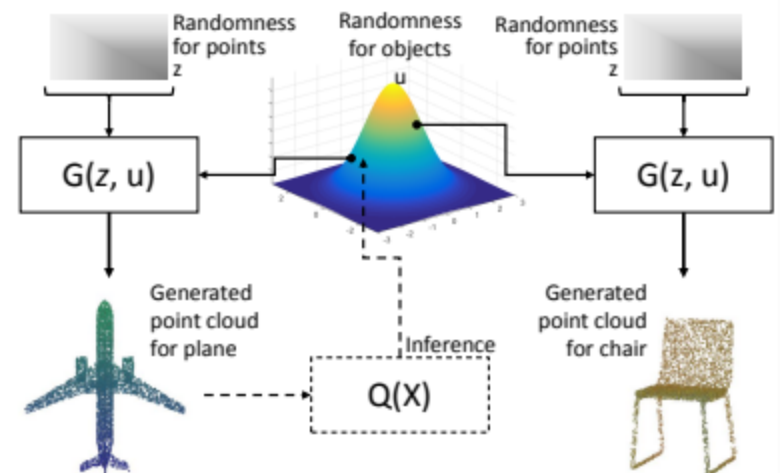


Figure 3: Overview of PC-GAN.

Synthetic Medical Images from Dual Generative Adversarial Networks

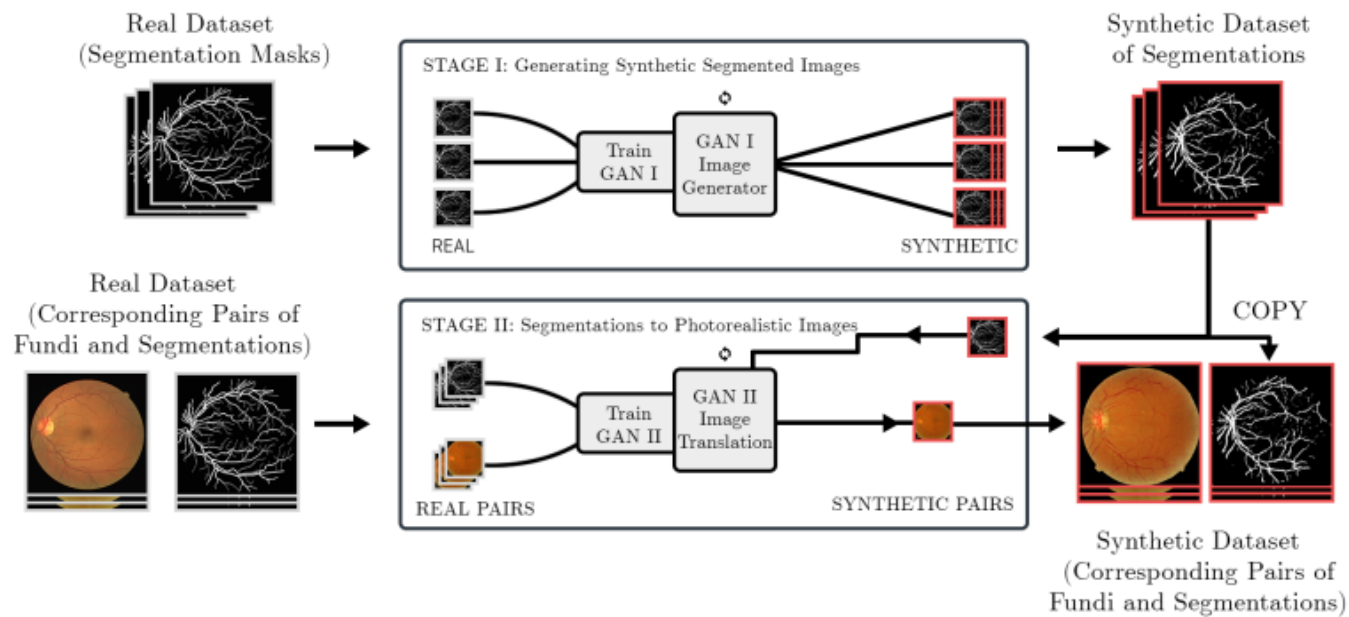
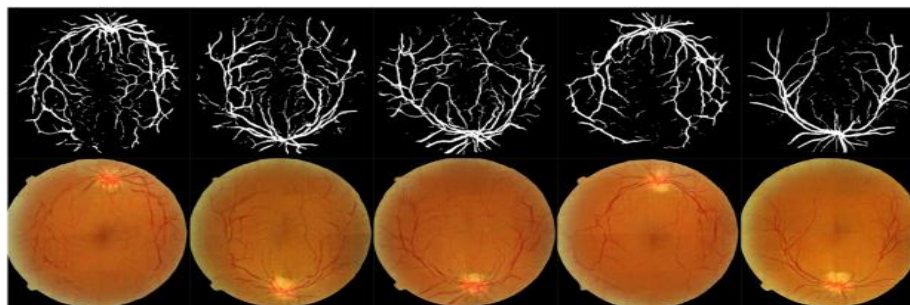


Figure 2: Flowchart of proposed pipeline.



Common training failures

Mode collapse. The generator learns to produce only a narrow subset of outputs — for example, always the same digit regardless of the input noise. It has found a single point that fools the discriminator and gets stuck there, ignoring the true diversity of the data distribution.

Vanishing gradients. When the discriminator becomes too good too quickly, it outputs near-zero probabilities for all generated samples. The gradient that flows back to the generator becomes vanishingly small, so G stops learning effectively. This is particularly severe with the original cross-entropy loss.

Training instability. G and D are optimized simultaneously but with competing objectives, so the training dynamics can oscillate or diverge rather than converge. A common symptom is loss curves that spike or collapse suddenly rather than decreasing smoothly.

Common training failures

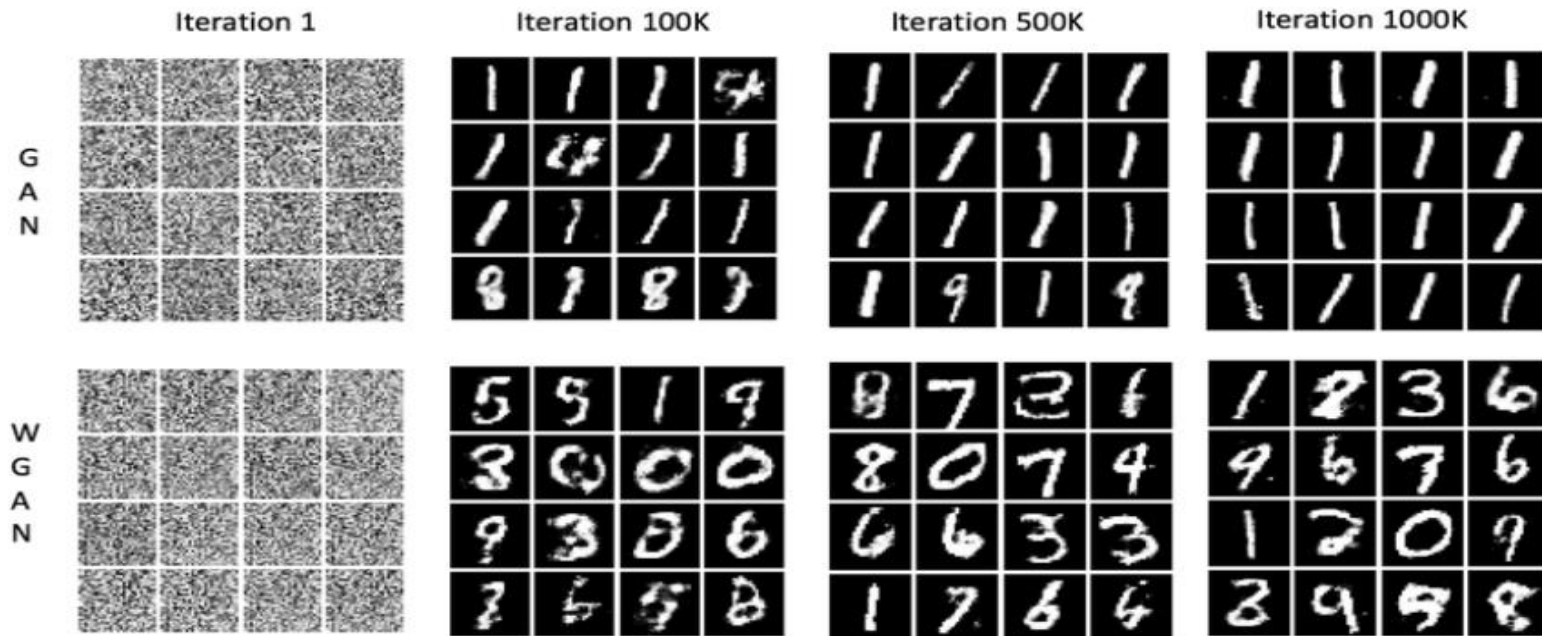
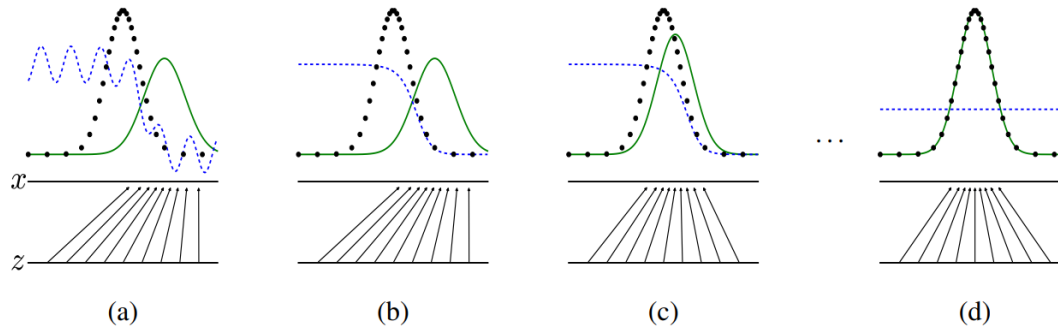


Figure from: <https://medium.com/@miraytopal/what-is-mode-collapse-in-gans-d3428a7bd9b8>

Stabilization Techniques

WGAN — Wasserstein GAN (Arjovsky et al., 2017). Replaces the original cross-entropy loss with the Wasserstein-1 distance, which measures how much "work" is needed to transform one distribution into another. This provides meaningful gradients even when the real and generated distributions do not overlap at all — the main cause of vanishing gradients in the original GAN. The discriminator (called a *critic* in WGAN) no longer outputs a probability but an unbounded score, and its weights are clipped to enforce a Lipschitz constraint. WGAN-GP (gradient penalty) later replaced weight clipping with a softer and more stable penalty term.

Spectral normalization (Miyato et al., 2018). Normalizes the weight matrices of the discriminator by their largest singular value at each layer, keeping the discriminator's Lipschitz constant under control without clipping. This prevents D from growing too powerful too fast, maintaining a useful learning signal for G throughout training. It is cheap to compute and easy to plug into any architecture.

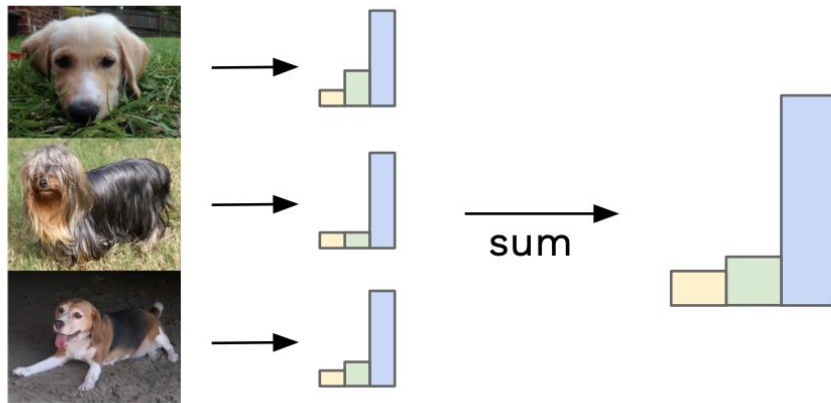
Minibatch discrimination. Allows the discriminator to look at multiple samples simultaneously rather than evaluating each one in isolation. It computes a similarity statistic across the batch and appends it as an extra feature, so D can detect when G is producing near-identical outputs. This directly penalizes mode collapse by making it visible to the discriminator.

Feature matching. Instead of training G to maximize D's output directly, G is trained to match the mean activations of real data in an intermediate layer of D. This stabilizes training by giving G a more consistent target that does not shift as rapidly as D's final output.

Evaluation Metrics

Inception Score — IS (Salimans et al., 2016). Feeds generated images through a pretrained Inception network and measures two things: each image should be confidently classified as something (low entropy per image), and the set of generated images should cover many different classes (high entropy across images). A high IS means the model generates both diverse and recognizable samples. Main limitation: it does not compare against real data at all, so a model can score well while still being far from the true distribution.

Similar labels sum to give focussed distribution



Different labels sum to give uniform distribution

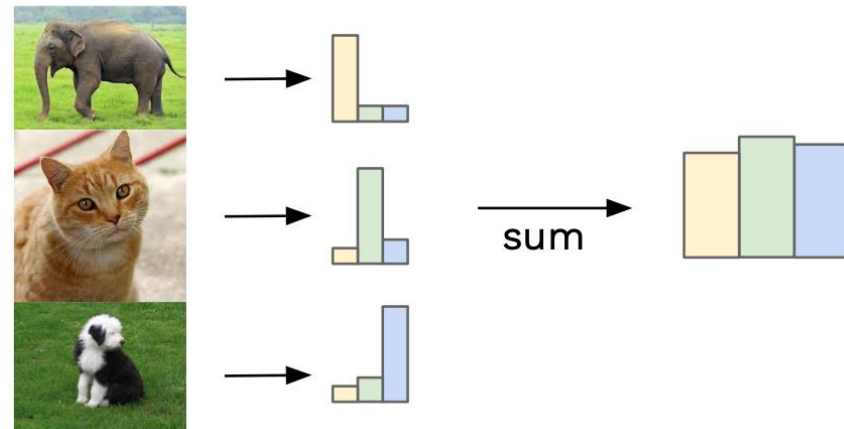


Figure from: <https://medium.com/octavian-ai/a-simple-explanation-of-the-inception-score-372dff6a8c7a>

Evaluation Metrics

Fréchet Inception Distance — FID (Heusel et al., 2017). Extracts feature vectors from both real and generated images using an Inception network, then fits a multivariate Gaussian to each set and measures the Fréchet distance between them. A lower FID means the generated distribution is closer to the real one in feature space, capturing both quality and diversity. FID is currently the most widely used metric — but it requires a large number of samples (typically 50k) to be stable, and it inherits the biases of the Inception network it relies on.

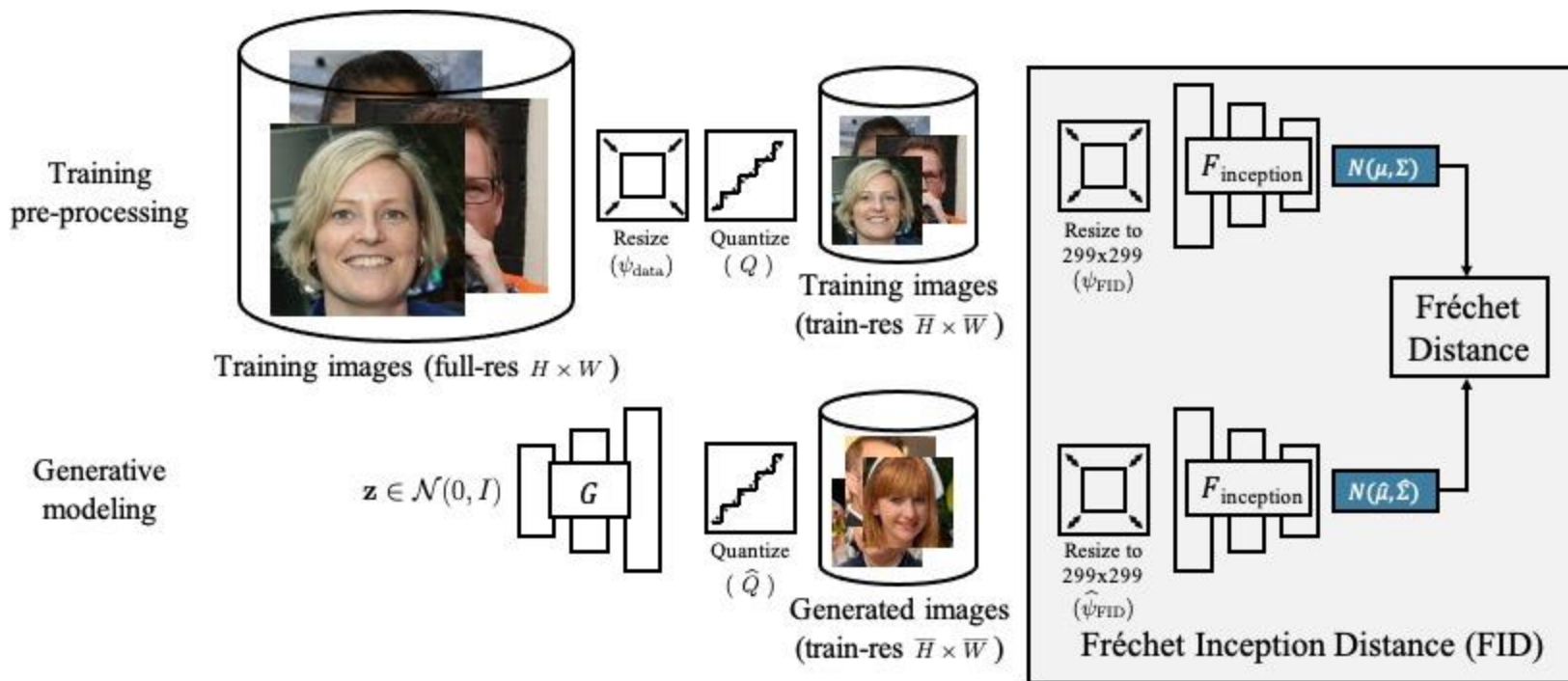


Figure from: <https://www.cs.cmu.edu/~clean-fid/>

Evaluation Metrics

Precision and Recall (Kynkäänniemi et al., 2019). Separates two questions that FID conflates into one number. Precision measures fidelity: what fraction of generated samples fall within the real data manifold? Recall measures coverage: what fraction of the real data manifold is covered by generated samples? A model with high precision but low recall generates realistic but repetitive images — a sign of mode collapse. A model with high recall but low precision covers the data well but produces low-quality outputs. Together they give a clearer diagnosis than a single scalar metric.

"No single metric captures everything. In practice, FID is the standard benchmark, IS is reported for comparison with older work, and Precision/Recall is used when diagnosing specific failure modes like mode collapse."

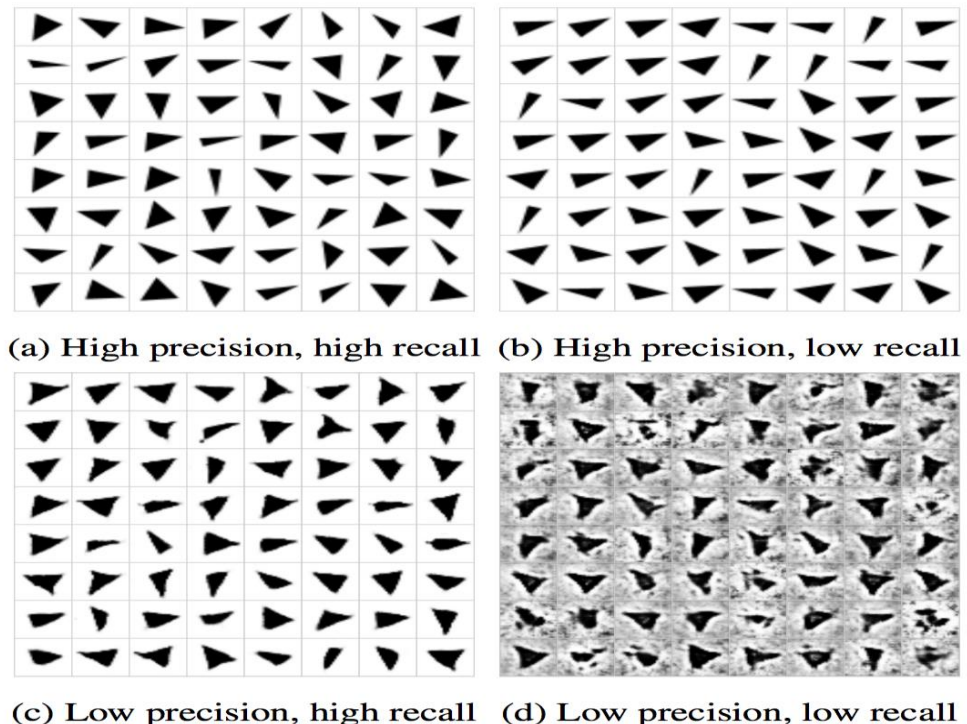


Figure from: <https://jonathan-hui.medium.com/gan-how-to-measure-gan-performance-64b988c47732>

Exercise. Search Papers in Conferences

Search GAN Papers in Conferences such as CVPR, NeurIPS, ICCV, ECCV, ICML, ICPR, BMVC, 3DV, etc. Then, enumerate the pros and cons of each method.

- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative Adversarial Networks. *Science Robotics*, 3(January), 2672–2680. <https://arxiv.org/pdf/1406.2661>
- Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. *4th International Conference on Learning Representations, ICLR 2016 - Conference Track Proceedings*. <https://arxiv.org/pdf/1511.06434>
- Mirza, M., & Osindero, S. (n.d.). *Conditional Generative Adversarial Nets*.
- Tahmid, M., Alam, M. S., Rao, N., & Ashrafi, K. M. A. (2016). Image-to-Image Translation with Conditional Adversarial Networks. *Proceedings of 2023 IEEE 9th International Women in Engineering (WIE) Conference on Electrical and Computer Engineering, WIECON-ECE 2023*, 468–472. <https://doi.org/10.1109/WIECON-ECE60392.2023.10456447>
- Zhu, J. Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. *Proceedings of the IEEE International Conference on Computer Vision, 2017-October*, 2242–2251. <https://doi.org/10.1109/ICCV.2017.244>
- Li, C. L., Zaheer, M., Zhang, Y., Póczos, B., & Salakhutdinov, R. (2018). Point Cloud GAN. *Deep Generative Models for Highly Structured Data, DGS@ICLR 2019 Workshop*. <https://arxiv.org/pdf/1810.05795>
- Guibas, J. T., Viridi, T. S., Li, P. S., Henry, S., & Gunn, M. (2017). *Synthetic Medical Images from Dual Generative Adversarial Networks*. <https://arxiv.org/pdf/1709.01872>